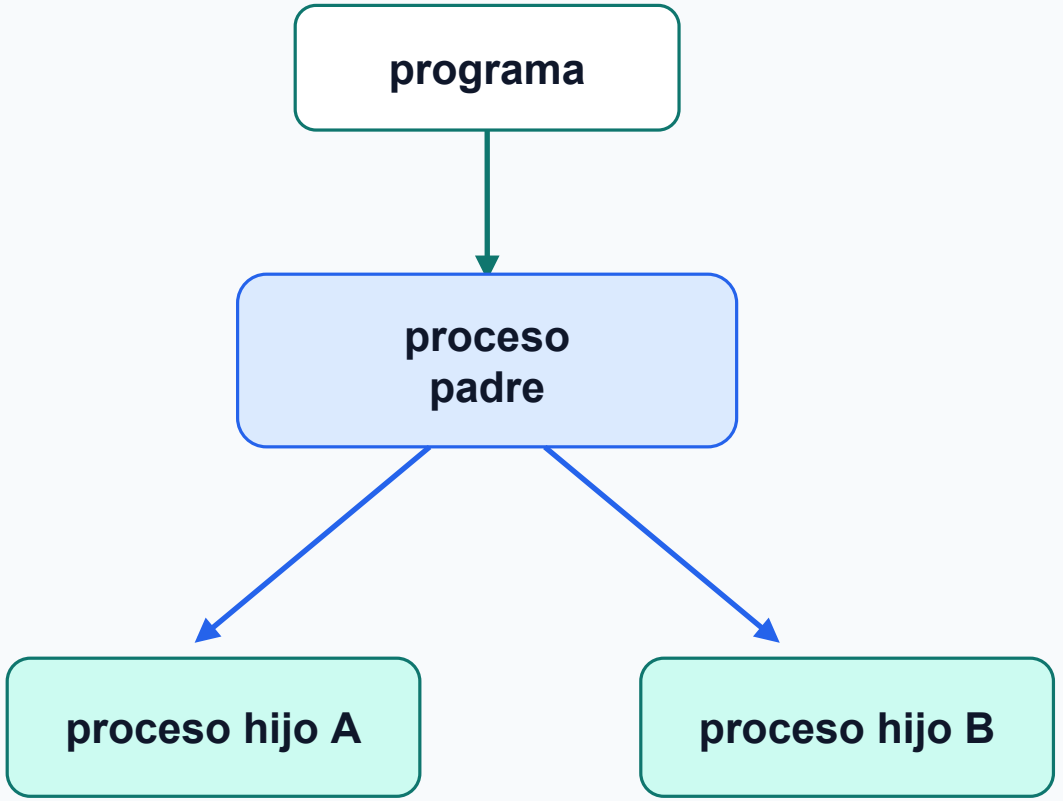
PSP · 1.ª evaluación

Concurrencia y procesos

del programa a ProcessBuilder

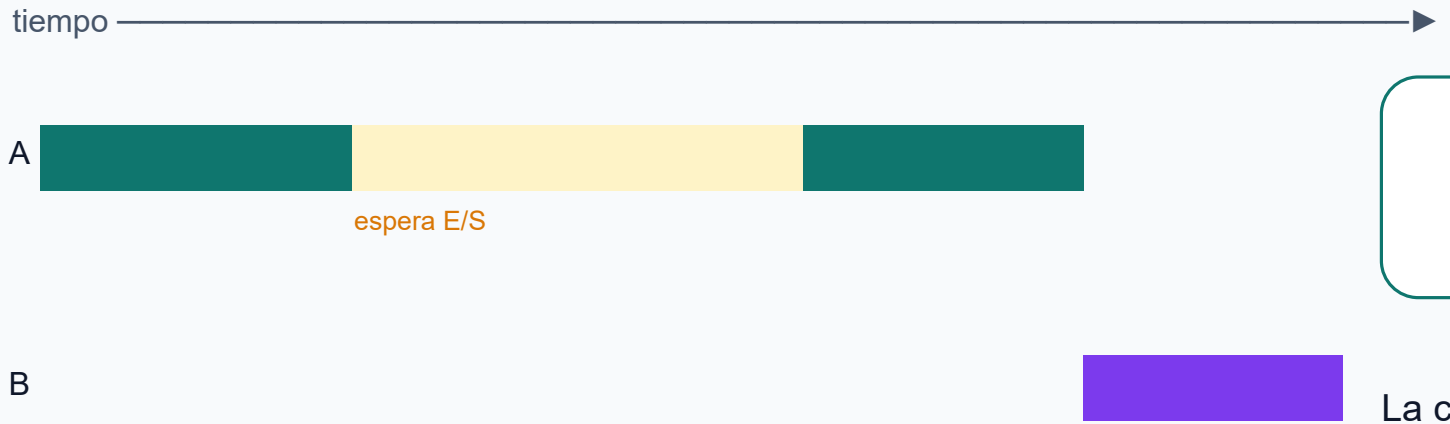
Hoy construiremos el modelo mínimo para:
• distinguir programa, ejecutable, proceso, servicio e hilo;
• entender concurrencia, paralelismo y distribución
• lanzar procesos hijo desde Kotlin/JVM
• comunicarnos con ellos y recoger sus resultados
• decidir cuándo merece la pena introducir concurrencia

Resultado final
un programa Kotlin/JVM que lanza
dos procesos hijo y recoge resultados



Kotlin/JVM · sin Android

Una tarea espera… ¿todo tiene que esperar?



Mientras A no termina, B ni siquiera empieza.

**Si A está esperando y B es independiente,
¿por qué no dejar que B progrese?**

La concurrencia puede aportar valor cuando existen:

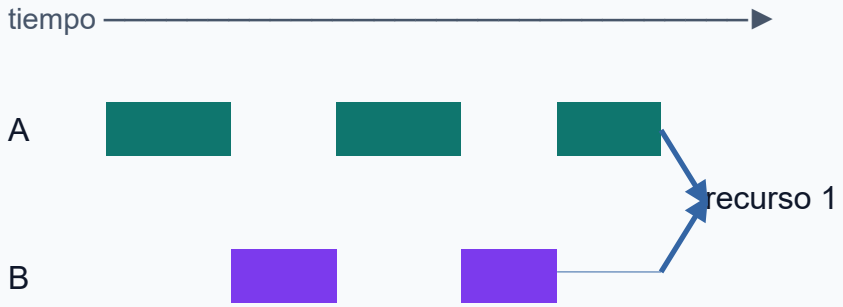
- tareas independientes
- esperas de E/S
- necesidad de mantener capacidad de respuesta
- trabajo que puede solaparse

Pero añade coordinación, complejidad y nuevos posibles errores.

Concurrencia no significa simultaneidad

CONCURRENCIA

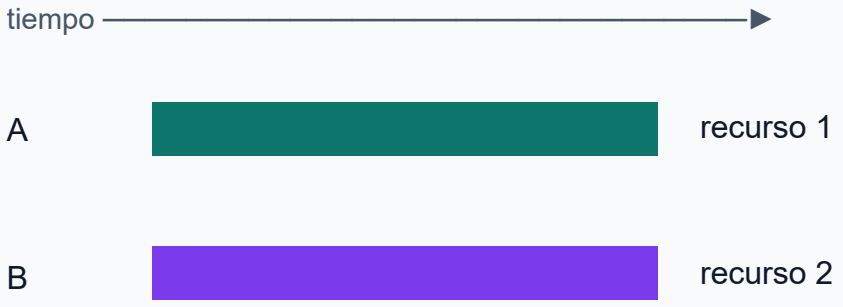
Varias tareas progresan durante un mismo intervalo temporal.



A y B progresan durante el mismo intervalo, aunque no se ejecuten simultáneamente.

PARALELISMO

Dos o más tareas se ejecutan realmente al mismo tiempo.



La concurrencia no requiere paralelismo.

Paralelismo y distribución responden a preguntas diferentes

	Paralelismo	Distribución
Pregunta	¿Se ejecutan tareas realmente a la vez?	¿Se ejecutan componentes en máquinas diferentes?
Característica	Ejecución simultánea	Máquinas conectadas mediante red
Puede aportar	Menor tiempo en trabajo paralelizable	Reparto de carga, escalabilidad, disponibilidad
Costes	Coordinación, reparto, acceso a datos	Comunicación, latencia, fallos parciales, complejidad
Escenario	Dividir un cálculo independiente	Varios equipos colaborando en un sistema

No son tres alternativas excluyentes (concurrencia, paralelismo y distibución)



Un sistema distribuido puede ser además:

- concurrente
- paralelo
- ambas cosas

¿Cuándo merece la pena introducir concurrencia?

Claramente sí

Dos tareas independientes pasan buena parte del tiempo esperando E/S.

→ Podemos solapar trabajo o esperas.

Claramente no

B necesita inmediatamente el resultado completo de A y ambas tareas son pequeñas.

→ La concurrencia no elimina la dependencia.

Depende

Un cálculo grande puede dividirse en partes independientes.

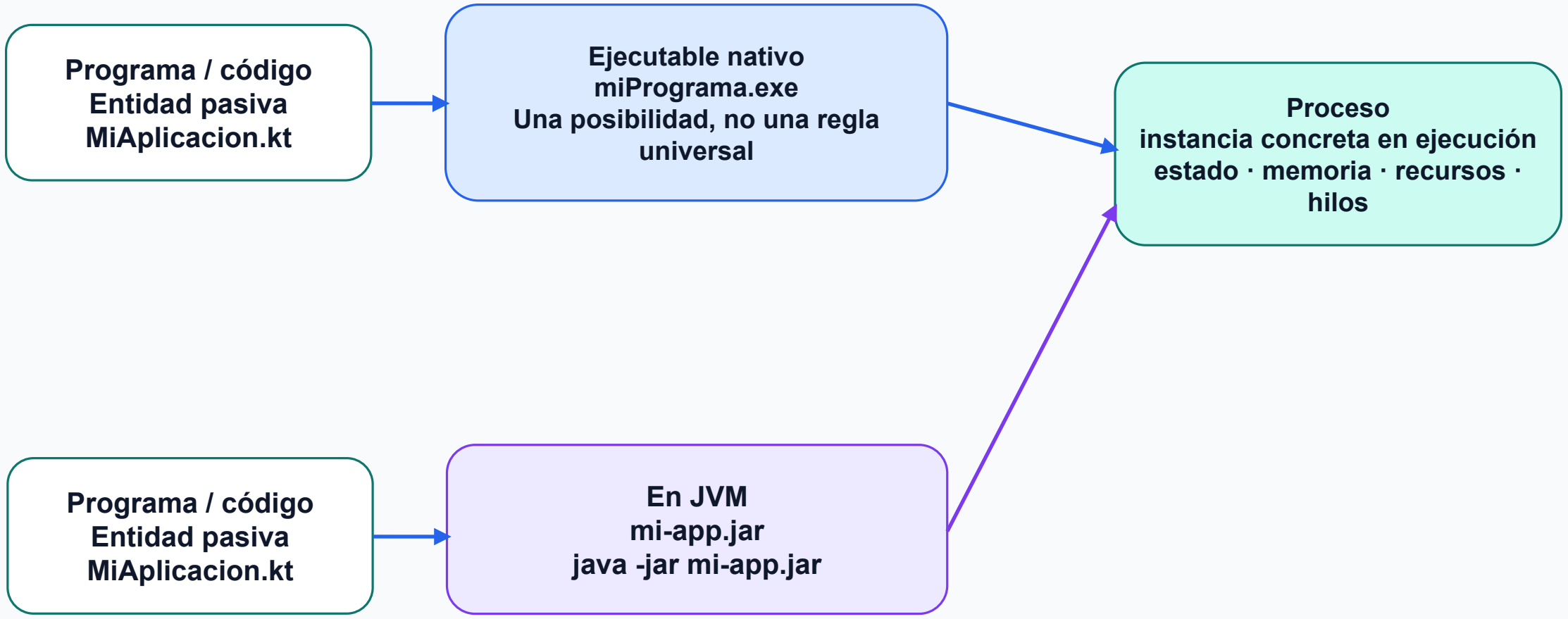
→ Depende del tamaño, recursos y coste de coordinación.

Más concurrencia ≠ programa mejor

Preguntas útiles:

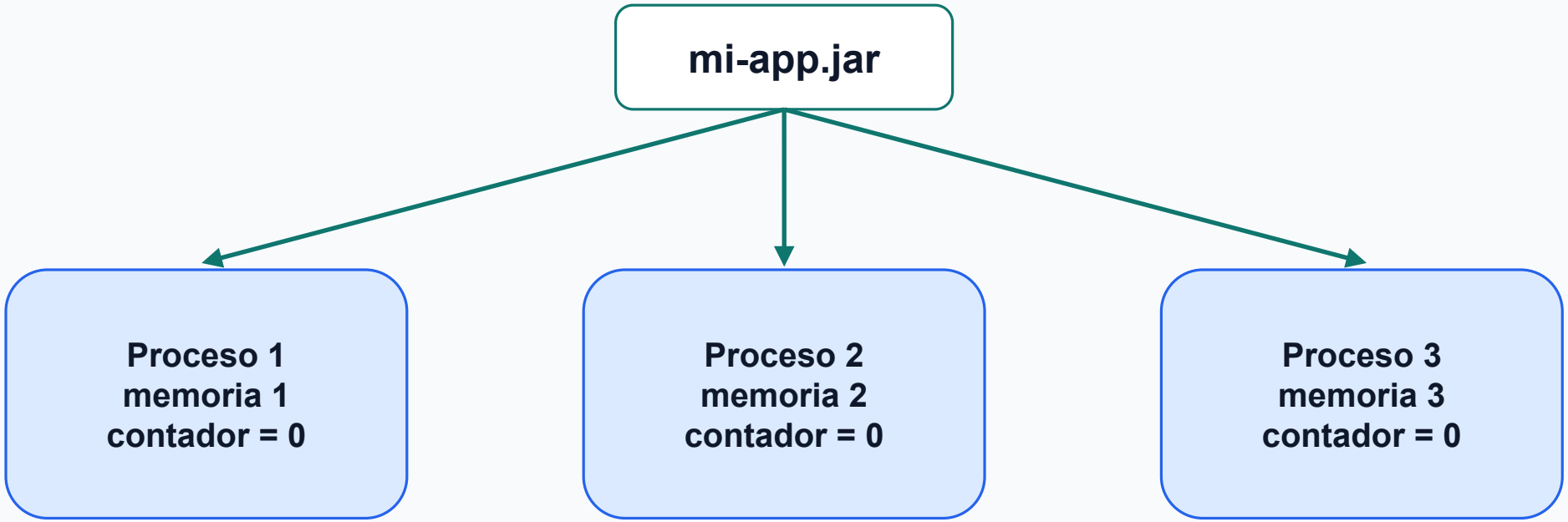
- ¿Son independientes las tareas?
- ¿Hay esperas?
- ¿Necesitamos capacidad de respuesta?
- ¿Podemos solapar trabajo?
- ¿Qué coordinación añadimos?
- ¿Qué nuevos errores pueden aparecer?

Programa, ejecutable y proceso no son lo mismo



Código almacenado ≠ ejecución activa

Un programa puede tener varias ejecuciones independientes



Los tres pueden ejecutar código del mismo programa, pero son instancias diferentes:

- estado propio
- recursos propios
- espacios de ejecución separados

Una variable Kotlin de un proceso no es automáticamente la misma variable de otro.

Proceso, hilo y servicio: relacionados, pero distintos



Hilo:
Flujo de ejecución independiente dentro de un proceso.

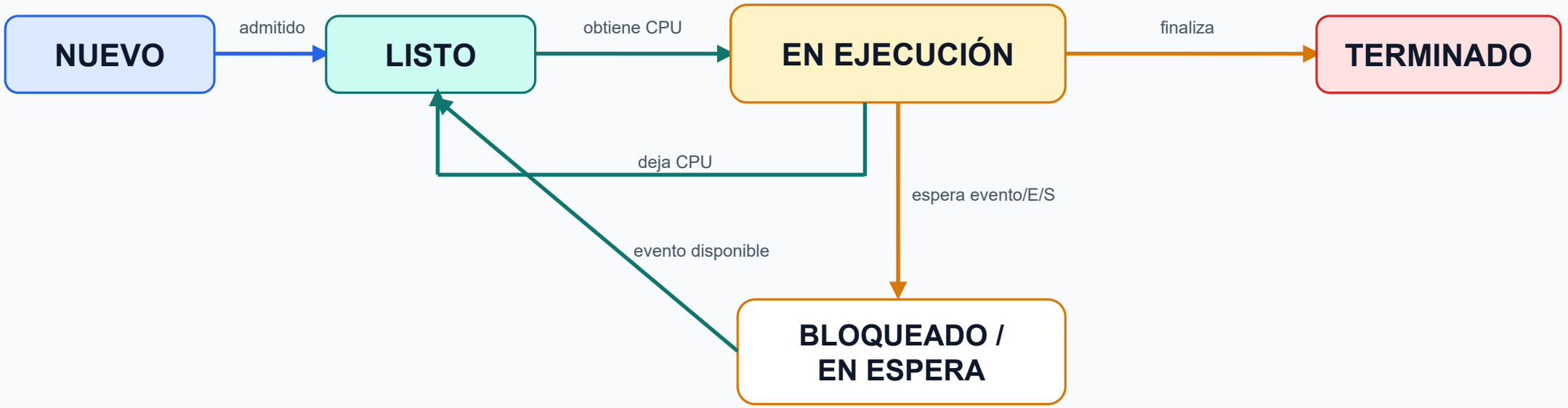
- un proceso contiene uno o más hilos
- los hilos de un proceso comparten recursos de ese proceso

Procesos diferentes presentan mucho mayor aislamiento entre sus espacios de ejecución.

Servicio:
funcionalidad o componente gestionado que presta una función continuada o bajo demanda.
Puede apoyarse en uno o más procesos.

Servicio ≠ proceso ≠ hilo

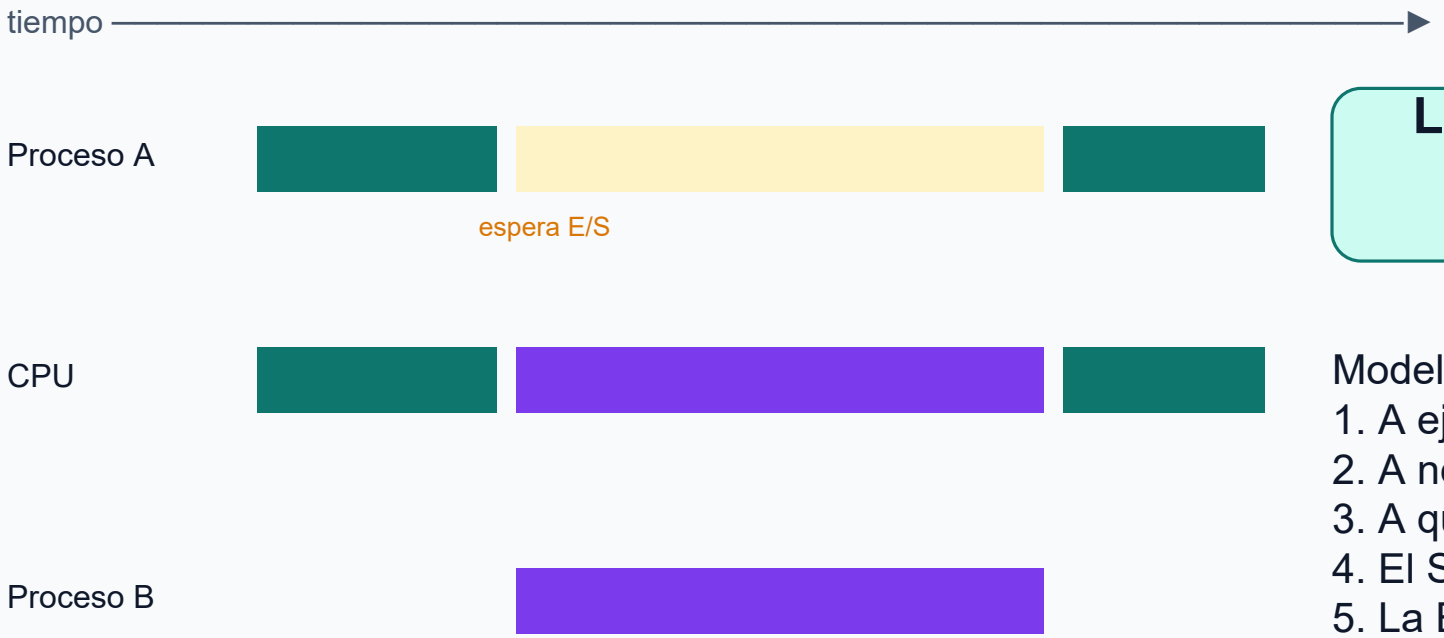
Estados de un proceso: modelo conceptual simplificado



Transiciones esenciales:
- listo → ejecución
- ejecución → listo
- ejecución → bloqueado
- bloqueado → listo
- ejecución → terminado

Modelo conceptual simplificado:
los SO reales pueden usar otros estados o nombres.

Mientras A espera, otro trabajo puede progresar



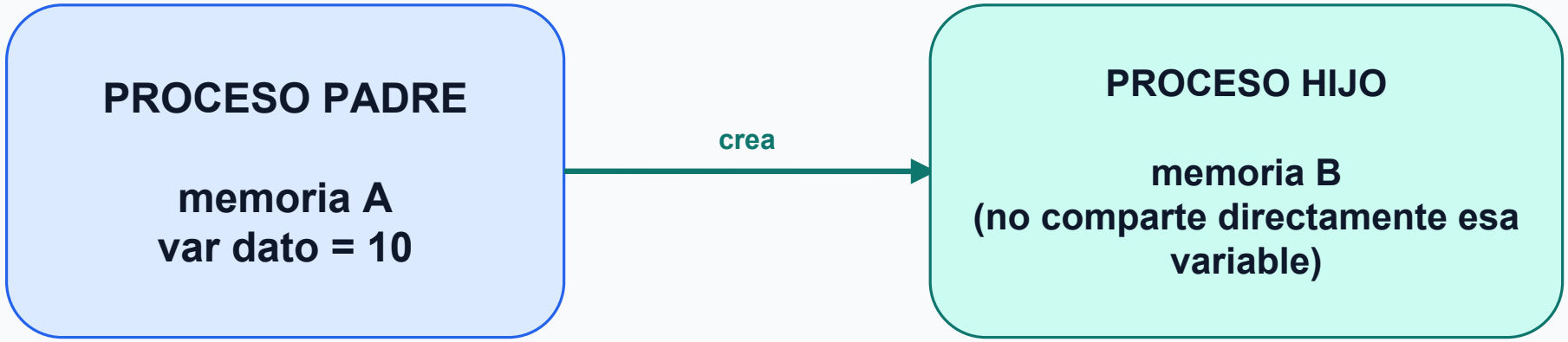
La espera de una tarea puede permitir que otra progrese.

Modelo mental:

1. A ejecuta.
2. A necesita E/S.
3. A queda en espera.
4. El SO puede ejecutar B.
5. La E/S termina.
6. A vuelve a estar listo.

No necesitamos dos núcleos para que exista concurrencia.

Un proceso puede crear otro proceso



Si quieren intercambiar información necesitan comunicación explícita.

Terminología
proceso hijo → proceso creado por otro proceso
hilo → flujo de ejecución independiente dentro de un proceso

No utilizamos “proceso hijo = hilo”.

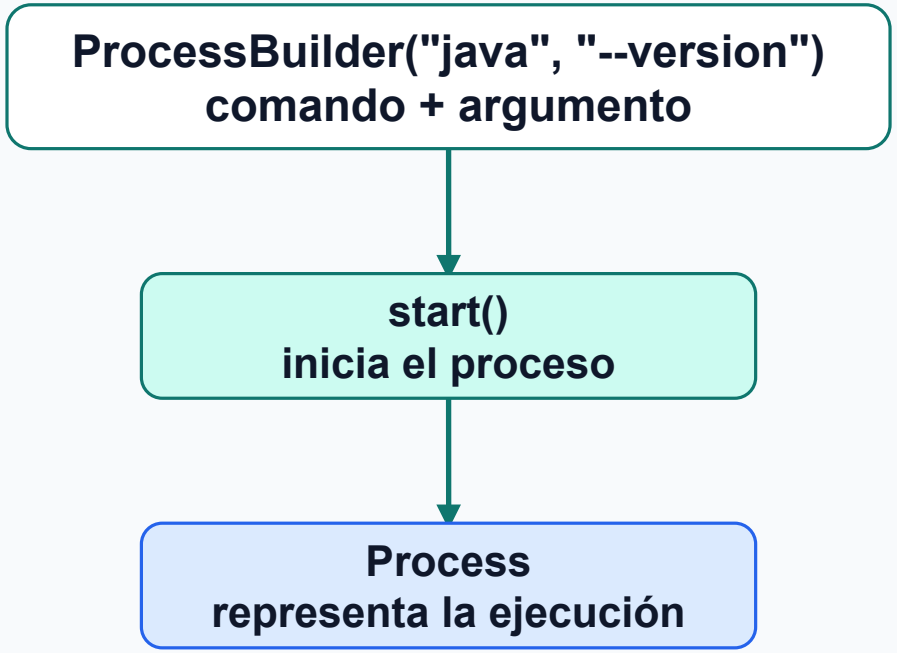
Nuestro primer proceso con ProcessBuilder

```
fun main() {
    val process = ProcessBuilder(
        "java",
        "--version"
    ).start()

    val exitCode = process.waitFor()

    val output = process.inputStream
        .bufferedReader()
        .use { it.readText() }

    println(output.trim())
    println("Código de salida: $exitCode")
}
```



- waitFor() espera su finalización
- obtenemos su código de salida
- el comando externo depende del entorno

Nota técnica: este proceso produce muy poca salida. Esperar antes de leer no es una receta general.

Lanzar, observar y esperar son operaciones diferentes

start()
Inicia el proceso y devuelve un Process.

```
val process = ProcessBuilder(
    "java",
    "--version"
).start()
```

isAlive
Consulta si en ese instante el proceso todavía no ha terminado. No espera.

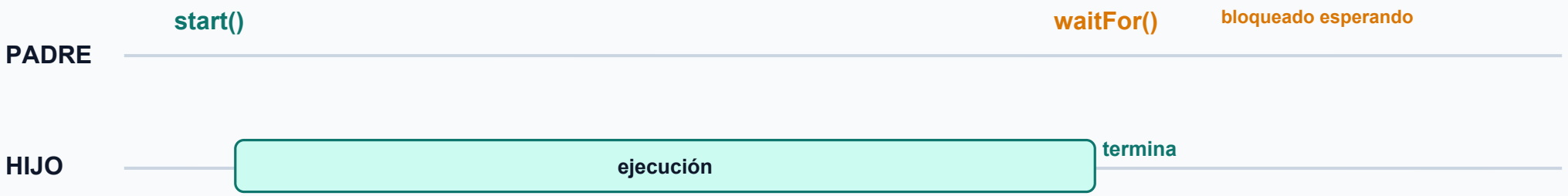
```
process.isAlive
```

En un proceso muy breve puede devolver ya false.

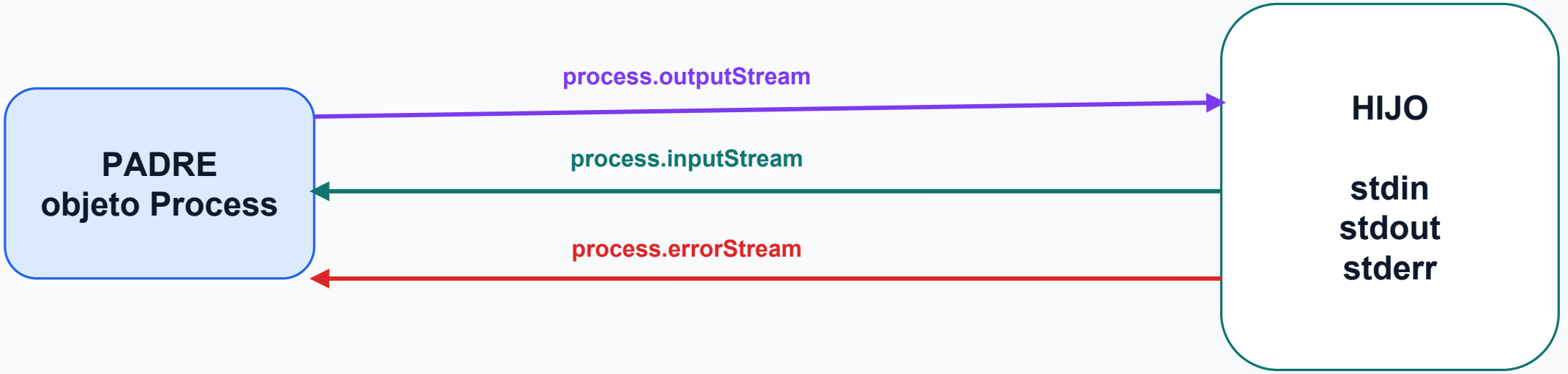
waitFor()
Espera, si hace falta, hasta que el proceso termine. Devuelve el código de salida.

```
val exitCode = process.waitFor()
```

Lanzar ≠ observar ≠ esperar



Padre e hijo: stdin, stdout y stderr



Queremos…

- escribir al stdin del hijo
- leer stdout del hijo
- leer stderr del hijo

Usamos…

process.outputStream
process.inputStream
process.errorStream

Comunicación ≠ espera
streams → datos
waitFor() → terminación

Demostración: comunicarnos con un hijo Kotlin/JVM

```kotlin
// ChildWorker.kt
fun main(args: Array<String>) {
    val name = args[0]
    val text = readln()

    println("$name -> "+ text.uppercase())
    System.err.println("$name: fin")
}
```

Para este ejemplo, ChildWorker.kt se entrega ya compilado como:
ChildWorker.jar

Se le puede llamar desde terminal:
java -jar ChildWorker.jar A

Resultado
stdout -> A -> HOLA
stderr -> A: fin
código -> 0

```kotlin
val process = ProcessBuilder(
    "java", "-jar", "ChildWorker.jar", "A"
).start()

process.outputStream.bufferedWriter().use {
    it.write("hola\n")
}

val code = process.waitFor()
val out = process.inputStream.bufferedReader().use { it.readLine() }
val err = process.errorStream.bufferedReader().use { it.readLine() }

println("stdout -> $out")
println("stderr -> $err")
println("código -> $code")
```

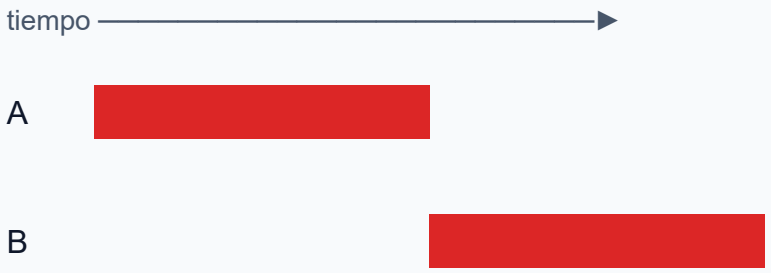
Este hijo produce solo dos líneas: esperar antes de leer es seguro aquí por diseño.

El error que elimina la concurrencia

✕ Serialización accidental

```
val a = launchChild("A")
a.waitFor()

val b = launchChild("B")
b.waitFor()
```

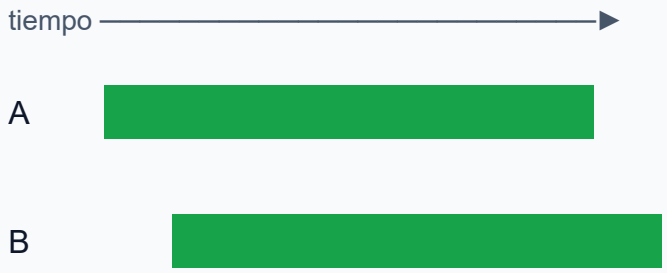


A termina antes de que B sea creado.

✓ Lanzar primero, esperar después

```
val a = launchChild("A")
val b = launchChild("B")

a.waitFor()
b.waitFor()
```



A y B pueden existir y progresar durante el mismo intervalo.

Lanza ambos antes de empezar a esperarlos.
Eso permite concurrencia. No demuestra paralelismo físico.

Micropráctica: dos procesos antes de esperar

Dispones ya de child-worker.jar y estas dos funciones:

```
fun launchChild(name: String): Process =
    ProcessBuilder(
        "java", "-jar", "child-worker.jar", name
    ).start()

fun send(process: Process, text: String) {
    process.outputStream.bufferedWriter().use {
        it.write("$text\n")
    }
}
```

Tu tarea
1. Crea el proceso A.
2. Crea el proceso B.
3. Consulta a.isAlive y b.isAlive.
4. Envía A ← "primero" y B ← "segundo".
5. Solo ahora espera A y B.
6. Lee stdout y stderr de cada uno.
7. Muestra los dos códigos de salida.

Hacer
start A
start B

---

wait A
wait B

Evitar
start A
wait A
start B
wait B

Puesta en común: solución mínima

```
fun main() {
    val a = launchChild("A")
    val b = launchChild("B")

    println("A vivo: ${a.isAlive}")
    println("B vivo: ${b.isAlive}")

    send(a, "primero")
    send(b, "segundo")

    val codeA = a.waitFor()
    val codeB = b.waitFor()

    val outA = a.inputStream.bufferedReader().use { it.readLine() }
    val errA = a.errorStream.bufferedReader().use { it.readLine() }
    val outB = b.inputStream.bufferedReader().use { it.readLine() }
    val errB = b.errorStream.bufferedReader().use { it.readLine() }

    println("A -> $outA | $errA | código $codeA")
    println("B -> $outB | $errB | código $codeB")
}
```

Salida esperada

A vivo: true
B vivo: true

A stdout -> A -> PRIMERO
A stderr -> A: fin
A código -> 0

B stdout -> B -> SEGUNDO
B stderr -> B: fin
B código -> 0

A aparece primero porque el padre imprime primero A. Eso no demuestra que A terminase antes que B.

No basta con que «haya salido algo»

1. ¿Qué procesos existen?
1 padre + 2 hijos

2. ¿Se lanzaron antes de esperar?
start A · start B

wait A · wait B

3. ¿Qué resultados produjeron?
stdout · stderr · códigos

4. ¿Han terminado?
Después de waitFor():
process.isAlive == false

5. ¿Qué podemos afirmar?
✓ coexistieron como procesos
✓ pudieron progresar concurrentemente
✗ no deducimos paralelismo físico

Documenta la intención
“A y B son tareas independientes.
Se lanzan ambos antes de esperar sus resultados.”

¿Secuencial, proceso o —más adelante— hilo?

Caso A
Calcular un valor y usar inmediatamente ese resultado en un segundo cálculo pequeño.

→ Secuencial

Caso B
Ejecutar dos herramientas externas independientes y recoger sus resultados.

→ Procesos

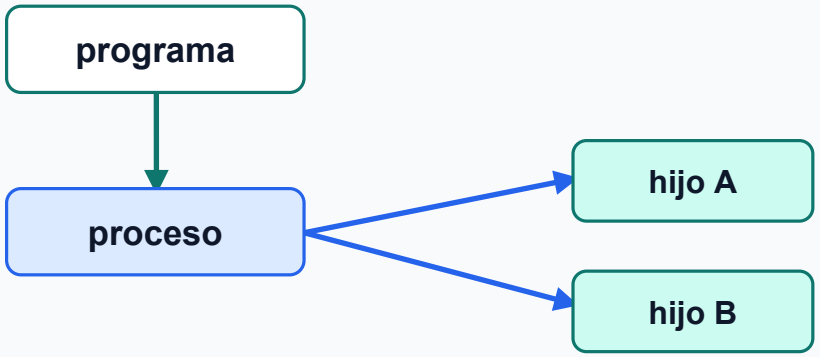
Caso C
Misma aplicación: dos tareas necesitan trabajar con los mismos objetos en memoria.

→ Hilos… más adelante

Caso D
Trabajos independientes pero tan pequeños que coordinar cuesta más.

→ Secuencial

Lo que ya sabemos



comunicación + espera + resultados

Siguiente paso
Si varias tareas pertenecen al mismo proceso y necesitan compartir directamente sus datos, entramos en el terreno de los hilos.

Ahí empieza P2.